

Customer Journey Map

Incident Management – Client Scripts & UI Policies

Project: Implement Client Script & UI Policy (Incident)

SCENARIO

An end-user or IT agent creates, updates, and resolves an Incident in ServiceNow. This journey maps every touchpoint where Client Scripts and UI Policies improve usability, enforce data quality, reduce errors, and guide the user through the Incident lifecycle.

Journey Phases

1. Entice / Report	2. Enter / Capture	3. Engage / Work	4. Exit / Resolve	5. Extend / Close
How does the user become aware and decide to log an Incident?	What does the user experience while filling the Incident form?	Core moments while the Incident is being worked on.	What happens when the Incident is resolved?	What happens after closure and follow-up?

Purpose of this Document

This Customer Journey Map supports the project **Implement Client Script & UI Policy (Incident)**. It identifies pain points and opportunities across the Incident lifecycle where Client Scripts and UI Policies deliver the highest value.

- Client Scripts – Client-side JavaScript (onLoad, onChange, onSubmit) that run in the browser to validate data, set field values, show messages, and control form behavior.
- UI Policies – Declarative rules that dynamically show/hide fields, make them mandatory or read-only based on conditions (without writing code).

Key Personas

End User / Requester	Service Desk Agent	Resolver / Technician	Incident Manager
Logs incidents via portal or form. Needs simple, guided experience.	Creates/updates incidents on behalf of users. Needs speed & accuracy.	Investigates and resolves. Needs clear fields & assignment logic.	Oversees process quality, SLAs, and reporting.

Detailed Customer Journey Map – Incident Form

Focus: ServiceNow Incident form experience and the role of Client Scripts & UI Policies at each step.

Steps – What does the person typically experience?

	ENTICE	ENTER		ENGAGE		EXIT	EXTEND	
Phase	Entice	Enter	Enter	Enter	Engage	Engage	Exit	Extend
Steps	Discover need User experiences an issue and decides to report it via portal, email, or phone.	Open Incident form Navigate to Create New Incident or open existing record.	Fill core fields Caller, Category, Subcategory, Short Description, Description, Impact/Urgency.	Save / Submit Submit the form. Number is generated. Assignment may occur.	Update & work Add work notes, change state, reassign, attach CI, update priority.	Investigate Resolve root cause, communicate with user, update resolution notes.	Resolve Set State = Resolved, fill Resolution Code & Notes, close related tasks.	Close & follow-up Confirm with user, close Incident, capture survey / lessons learned.
Interactions (People / Places / Things)	Service Portal / Mobile / Email / Phone Knowledge base, Chatbot	Incident form (UI) Browser / Agent Workspace	Form fields, Reference pickers, Choice lists Client Scripts & UI Policies active	Save button, onSubmit scripts Business rules (server)	Work notes, Activity stream, Related lists Assignment group members	CI relationships, Knowledge, Collaboration Resolver + Caller	Resolution fields, Close notes UI Policy: mandatory on Resolved	Survey, Email notification, Dashboard Incident Manager view
Goals & Motivations	<i>“Help me report this quickly so someone can fix it.”</i>	<i>“Help me open the right form with minimal clicks.”</i>	<i>“Help me fill only what is needed and avoid mistakes.”</i>	<i>“Help me submit successfully without errors.”</i>	<i>“Help me keep the record accurate and up to date.”</i>	<i>“Help me resolve this correctly and document it.”</i>	<i>“Help me close cleanly with proper resolution data.”</i>	<i>“Help me confirm satisfaction and improve next time.”</i>
Positive Moments	Clear self-service options and knowledge articles reduce ticket volume.	Form opens fast; relevant fields pre-populated from user profile.	Dynamic field visibility makes the form feel simple and guided.	Instant confirmation number and clear next-step message.	Work notes auto-notify relevant parties; state changes are intuitive.	Resolution fields appear only when needed; good suggestions available.	Clean mandatory checks prevent incomplete resolutions.	Survey is short and optional; historical data is easy to find.
Negative Moments	User doesn't know whether to raise Incident or Request.	Too many fields visible on load; form feels overwhelming.	Wrong Category chosen → wrong assignment; Priority calculation confusing.	Submit fails with unclear error; mandatory fields not highlighted.	State changes allowed that skip process steps; missing work notes.	Resolution Code free-text or inconsistent; no guidance.	Can resolve without filling Resolution Notes; data quality suffers.	No follow-up prompt; closed tickets hard to reopen if needed.
Areas of Opportunity (Client Script & UI Policy)	UI Policy / Portal: guide user to correct record type (Incident vs Request).	onLoad Client Script: set default values, hide non-essential sections. UI Policy: show only relevant fields for role.	onChange Client Script: auto-set Priority from Impact + Urgency. UI Policy: make Subcategory mandatory when Category is set.	onSubmit Client Script: validate mandatory data & show clear messages. Prevent save if critical fields empty.	UI Policy: control State transitions (hide invalid states). onChange: require Work Notes when State changes.	UI Policy: show Resolution fields only when State = Resolved. Client Script: suggest Resolution Codes.	UI Policy: make Resolution Code + Notes mandatory on Resolve. onSubmit: block resolve if notes missing.	UI Policy: read-only after Closed; allow reopen under conditions. Client Script: trigger survey or feedback prompt.

Implementation Recommendations

Based on the journey map above, the following Client Scripts and UI Policies are recommended for the Incident table (incident).

Recommended UI Policies

UI Policy Name	Condition	Actions (Fields)
Make Resolution fields mandatory	State is Resolved OR Closed	Resolution code → Mandatory Resolution notes → Mandatory Close notes → Mandatory (if Closed)
Show Resolution section only when needed	State is Resolved OR Closed	Resolution code → Visible Resolution notes → Visible (Hide otherwise)
Subcategory mandatory on Category	Category is not empty	Subcategory → Mandatory + Visible
Read-only after Closed	State is Closed	Most fields → Read-only (except specific reopen fields if allowed)
Hide Assignment fields for End Users	User has role = itil (or opposite for portal)	Assignment group / Assigned to → Visible only for ITIL users
Priority read-only (calculated)	Always (or when Impact/Urgency present)	Priority → Read-only (set by Client Script)

Recommended Client Scripts

Script Name	Type	Purpose / Logic
Set Priority from Impact & Urgency	onChange	When Impact or Urgency changes, calculate and set Priority using the standard Priority Matrix. Keep Priority field read-only via UI Policy.
Require Work Notes on State Change	onChange	When State changes to In Progress, On Hold, or Resolved, make Work notes mandatory (or show info message) so the agent documents the reason.
Validate on Submit	onSubmit	Block submission if State = Resolved and Resolution notes are empty. Display a clear g_form.addErrorMessage(). Return false to prevent save.
Populate Caller defaults onLoad	onLoad	If new record and current user is not ITIL, set Caller = current user and pre-fill Location / Contact from user profile.
Clear Subcategory on Category change	onChange	When Category changes, clear Subcategory so the user is forced to choose a valid combination (prevents orphaned subcategory values).
Confirm resolve action	onSubmit	If State is being set to Resolved, optionally show a confirmation dialog summarizing Resolution Code and Notes before allowing save.

Best Practices & Design Guidelines

Client Scripts

- 1. Prefer UI Policies over Client Scripts when the requirement is only show/hide, mandatory, or read-only. UI Policies are easier to maintain and do not require coding skills.
- 2. Keep Client Scripts focused and short. One responsibility per script improves readability and debugging.
- 3. Use `g_form` methods (`setValue`, `setMandatory`, `setDisplay`, `setReadOnly`, `addInfoMessage`, `addErrorMessage`) instead of direct DOM manipulation.
- 4. Always check `g_form.isNewRecord()` or field values before applying logic to avoid unexpected side-effects on existing records.
- 5. For `onSubmit` scripts, return `false` to abort the save and provide a clear user message explaining what is missing.
- 6. Test scripts in both the classic UI and Agent Workspace / Service Portal (if applicable) because client APIs can differ slightly.

UI Policies

- 7. Give every UI Policy a clear, descriptive name and short description so future administrators understand the intent.
- 8. Order UI Policies carefully when multiple policies affect the same field; higher order runs later.
- 9. Use “Reverse if false” thoughtfully so that when the condition is no longer true the previous state is restored.
- 10. Avoid conflicting policies (one policy makes a field mandatory while another hides it). Document dependencies.
- 11. Combine UI Policies with Data Policies (server-side) when the same rule must also be enforced on import, API, or mobile.

Success Metrics

Metric	Before (Baseline)	Target After Implementation
% of Incidents resolved with empty Resolution Notes	High (data quality gap)	< 2%
Form submission errors / rework rate	Frequent user confusion	Significant reduction via clear validation messages
Average time to correctly categorize an Incident	Longer due to wrong category/subcategory	Faster with guided mandatory fields & clear defaults
Agent satisfaction with Incident form usability	Form feels cluttered / inconsistent	Higher scores after progressive disclosure via UI Policies

Document Summary

This Customer Journey Map translates the Incident lifecycle into concrete opportunities for Client Scripts and UI Policies. By addressing the identified negative moments—especially around form complexity, mandatory resolution data, priority calculation, and state transitions—the project will improve data quality, reduce agent friction, and deliver a cleaner experience for both end users and IT staff.